

MODULE 1 — Dataset Validation & Preparation

Objective

Before training any model, make sure the 5,000-row dataset is actually usable.

You currently have:

india_scam_detection_master_5000.csv

The first job is **not model training**.

The first job is to determine:

"Can I trust this dataset enough to train a model on it?"

1.1 Load the dataset

Use:

- Python
- Pandas
- NumPy

Check:

number of rows
number of columns
data types
missing values
duplicate messages
unique messages

Expected:

Rows: 5000
Columns: 18

1.2 Check class distribution

You should get approximately:

SCAM	1500
LEGITIMATE	1500
SUSPICIOUS	1000
FALSE_MISINFORMATION	1000

Plot:

- bar chart
- pie chart

This becomes part of your project report.

1.3 Check language distribution

For example:

English
Hinglish
Hindi-Roman

Then check:

SCAM × LANGUAGE
LEGITIMATE × LANGUAGE

This is important because your system is supposed to work for Indian users.

1.4 Check scam types

Create a distribution:

KYC_FRAUD
OTP_THEFT
UPI_FRAUD
BANK_IMPERSONATION
COURIER_FRAUD
JOB_SCAM
INVESTMENT_SCAM
...

Look for categories with too few examples.

1.5 Check duplicate/template leakage

This is **very important**.

Suppose you have:

"Your KYC will expire today. Share OTP..."

and:

"Your KYC will expire tomorrow. Share OTP..."

If almost identical messages appear in both training and testing, your accuracy can become artificially high.

Therefore:

duplicate detection

near-duplicate detection

template detection

should happen before splitting the dataset.

1.6 Validate annotations

Check whether:

SCAM

actually corresponds to:

risk_level = HIGH

and whether:

asks_for_otp = 1

is actually supported by the message.

You don't want contradictory labels.

1.7 Create final datasets

At the end of Module 1:

```
raw_dataset.csv
↓
clean_dataset.csv
↓
train.csv
validation.csv
test.csv
```

I recommend:

70% → training
15% → validation
15% → testing

Important: split after removing duplicates/near-duplicates and use stratification by verdict.

Module 1 deliverable

You should have:

```
data/
├── raw/
│   └── master_5000.csv
├── processed/
│   └── cleaned_dataset.csv
├── train.csv
├── validation.csv
└── test.csv
```

MODULE 2 — NLP Preprocessing Pipeline

This deserves its own module.

Your raw WhatsApp message could be:

"URGENT!!! Your SBI a/c will be blocked 🤖. Click <http://xyz.com> NOW!!!"

The model shouldn't receive completely unprocessed text.

2.1 Text cleaning

Implement:

- lowercase conversion
- whitespace normalization
- unnecessary punctuation handling
- Unicode normalization
- emoji handling
- URL detection
- phone number detection
- currency detection
- OTP detection

But **don't blindly remove everything**.

For example:

OTP

₹50,000

http://...

UPI

KYC

are important fraud signals.

2.2 Feature extraction

Create features such as:

contains_url

contains_phone

asks_for_otp

asks_for_upi_pin

asks_for_money

impersonates_authority

urgency

These become useful to the Risk Engine.

2.3 Text representation

For the first version:

TF-IDF

Message

↓

TF-IDF



Numerical feature vector



ML model

Later, you can experiment with transformer embeddings.

Module 2 deliverable

Create a reusable:

preprocessor.py

that accepts:

message

and produces:

clean_text

linguistic_features

MODULE 3 — Verdict Classification

This is your **first ML model**.

Objective

Determine:

SCAM

LEGITIMATE

SUSPICIOUS

FALSE_MISINFORMATION

Input

message

Output

For example:

```
{  
  "verdict": "SCAM",
```

```
"probabilities": {  
  "SCAM": 0.91,  
  "LEGITIMATE": 0.03,  
  "SUSPICIOUS": 0.05,  
  "FALSE_MISINFORMATION": 0.01  
}
```

MODULE 4 — ML Model Comparison

Now train your four proposed algorithms.

Model A — Logistic Regression

TF-IDF

↓

Logistic Regression

↓

4-class prediction

Advantages:

- fast
 - interpretable
 - strong baseline
 - probability estimates
-

Model B — Linear SVM

TF-IDF

↓

Linear SVM

↓

Verdict

Very useful for text classification.

Model C — Naive Bayes

Excellent baseline for spam/text classification.

TF-IDF
↓
Naive Bayes
↓
Verdict

Model D — Random Forest

This is especially useful for your **structured risk features**, although raw TF-IDF + Random Forest may not necessarily outperform linear text models.

You can test:

TF-IDF features
+
structured features
↓
Random Forest

Model comparison

Create a table:

Model	Accuracy	Precision	Recall	F1
Logistic Regression				
Linear SVM				
Naive Bayes				
Random Forest				

Don't decide based only on accuracy.

For this project:

Recall for SCAM is extremely important.

A detector that says:

"99% accuracy"

but misses many actual scams isn't useful.

MODULE 5 — Scam Type Classification

Once the first model says:

SCAM

we need another model to answer:

What kind of scam is this?

Example:

Message



Verdict Classifier



SCAM



Scam Type Classifier



KYC_FRAUD

Possible classes:

KYC_FRAUD

OTP_THEFT

UPI_FRAUD

BANK_IMPERSONATION

COURIER_FRAUD

JOB_SCAM

INVESTMENT_SCAM

LOAN_SCAM

LOTTERY_SCAM

POLICE_IMPERSONATION

DIGITAL_ARREST

PHISHING

ACCOUNT_TAKEOVER

ELECTRICITY_SCAM

FASTAG_SCAM

TAX_REFUND_SCAM

SCHOLARSHIP_SCAM

Important

Don't train scam-type classification on legitimate messages.

The pipeline should be:

Message
↓
Verdict
↓
IF SCAM
↓
Scam Type

MODULE 6 — Risk Engine

This is one of your **novelty components**.

The ML model says what it thinks.

The Risk Engine combines:

ML prediction
+
behavioral indicators
+
message features
+
rule-based evidence

For example:

ML Scam probability	0.91
OTP request	TRUE
UPI PIN request	TRUE
Money request	TRUE
Urgency	HIGH
Authority impersonation	TRUE
Suspicious URL	TRUE

The engine converts these into:

LOW
MEDIUM
HIGH
CRITICAL

Example

SCAM probability = 91%

Risk indicators:

+ OTP request

- + UPI PIN request
- + payment request
- + authority impersonation
- + urgent language
- + external URL

Final:
CRITICAL / HIGH RISK

Why this is important

A pure ML classifier may say:

SCAM — 89%

Your Risk Engine can explain:

Why?

That is what makes your project more than a simple spam classifier.

MODULE 7 — Explainable Verdict Engine

This converts all the internal outputs into a human-readable answer.

Input:

ML prediction
scam type
risk score
risk indicators

Output:

 SCAM — HIGH RISK

Confidence: 94%

Type:
KYC / OTP Fraud

Why:

- Requests OTP
- Creates urgency
- Impersonates a bank
- Requests sensitive information

- Contains an external link

Recommended action:

- ✗ Do not share OTP
- ✗ Do not click the link
- ✗ Do not transfer money
- ✓ Verify through the official bank application

This should be **template/rule controlled**, not entirely generated by an LLM.

That makes your verdict deterministic and easier to evaluate.

An LLM can optionally be used later to make the explanation more natural, but it shouldn't be allowed to arbitrarily change the security verdict.

MODULE 8 — FastAPI Backend

Now we have a working detection engine.

We expose it through an API.

Recommended endpoints:

```
POST /api/v1/detect/message
POST /api/v1/detect/pdf
GET /api/v1/health
```

Message endpoint

Request:

```
{
  "message": "Your KYC will expire today. Send OTP immediately."
}
```

Response:

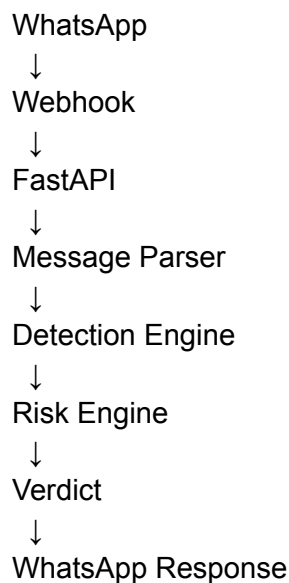
```
{
  "verdict": "SCAM",
  "risk_level": "HIGH",
  "confidence": 0.94,
  "scam_type": "KYC_FRAUD",
  "evidence": [
    "OTP request",
    "Urgency",
    "Authority impersonation"
  ]
}
```

```
],  
"recommended_action": "Do not share OTP"  
}
```

MODULE 9 — WhatsApp Integration

Only after the API works should you connect WhatsApp.

Architecture:



The webhook should handle:

text message
image/screenshot
PDF/document

Initially implement **text only**.

Then add PDF.

MODULE 10 — PDF Detection

This is your second major module.

Don't mix the PDF system into the text classifier.

Create a separate pipeline:

PDF
↓
File Validation
↓
PDF Extraction
↓
OCR if necessary
↓
Text Analysis
↓
URL Analysis
↓
QR Analysis
↓
Metadata Analysis
↓
Risk Engine
↓
PDF Verdict

10.1 PDF validation

Check:

file type
file size
page count
password protection
corruption

10.2 Text extraction

Use a PDF parser.

For text PDFs:

PDF
↓
Extract text

For scanned PDFs:

PDF
↓
Render pages

↓
OCR
↓
Text

10.3 Detect suspicious content

Look for:

KYC requests
OTP requests
UPI payment requests
bank details
Aadhaar/PAN requests
urgent payment
fake government notices
fake job offers
suspicious claims

10.4 URL extraction

Extract every URL:

PDF
↓
URLs
↓
Domain analysis

Check things like:

- suspicious domain patterns
- URL shorteners
- domain mismatch
- unusual TLD
- IP-address URLs
- lookalike domains

For production, integrate a reputable URL reputation service rather than making a verdict solely from string heuristics.

10.5 QR code analysis

This is potentially a **nice novelty feature** for your project.

Example:

PDF
↓
QR detector
↓
QR content
↓
URL / UPI / text
↓
Risk analysis

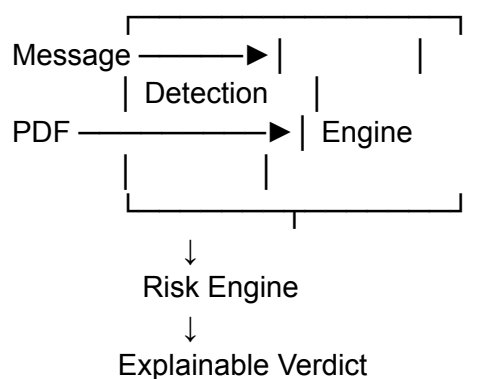
A PDF containing:

"Scan this QR to pay ₹5,000"

should be treated differently from a normal informational document.

MODULE 11 — Unified Risk Engine for Messages + PDFs

Eventually both systems feed the same risk engine.



So the system has a consistent verdict format.

MODULE 12 — Evaluation

This should be treated as a full project module, not something you do at the end for five minutes.

You need to evaluate **each component separately**.

12.1 Verdict classifier

Measure:

Accuracy
Precision
Recall
F1-score
Confusion Matrix

Especially:

SCAM recall
SCAM precision

12.2 Scam-type classifier

Measure:

Macro F1
Weighted F1
Per-class precision
Per-class recall
Confusion matrix

12.3 Risk engine

Create manually verified test cases:

CLEAR SCAM
CLEAR LEGITIMATE
AMBIGUOUS
HARD NEGATIVE
MISINFORMATION

Measure:

False Positive Rate
False Negative Rate
Critical Scam Miss Rate

12.4 PDF detector

Create a PDF test collection:

- legitimate PDFs
- suspicious PDFs
- scam PDFs
- scanned PDFs
- image-heavy PDFs
- PDFs with URLs
- PDFs with QR codes
- password-protected PDFs

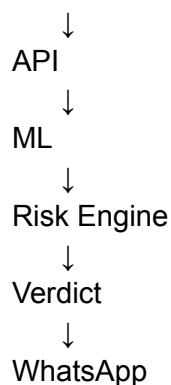
Measure:

- PDF detection accuracy
- URL detection accuracy
- QR extraction success
- OCR extraction success
- false positives
- false negatives

MODULE 13 — End-to-End Testing

Finally:

WhatsApp message



Test the complete workflow.

Example:

Input

"URGENT: Your SBI KYC has expired. Share OTP immediately or your account will be blocked."

Expected

VERDICT: SCAM

RISK: HIGH/CRITICAL

TYPE: KYC/OTP FRAUD

EVIDENCE:

OTP request

urgency

bank impersonation

RECOMMENDATION:

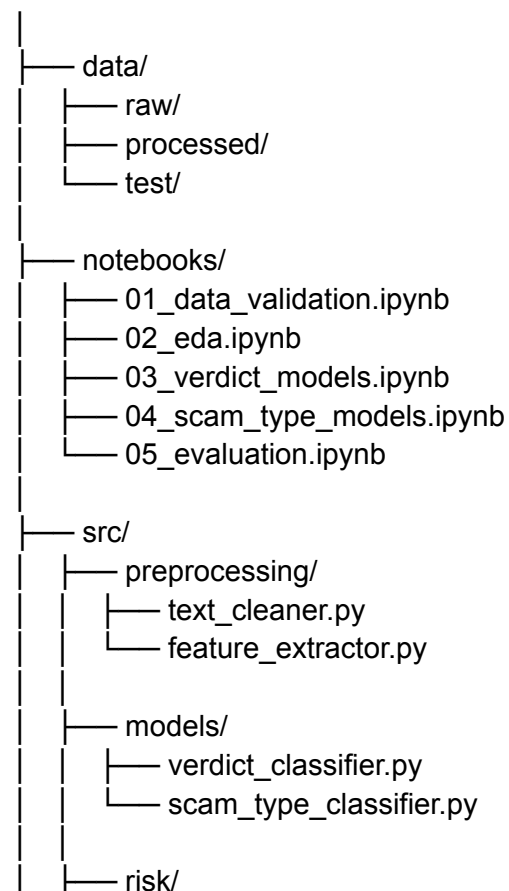
Don't share OTP

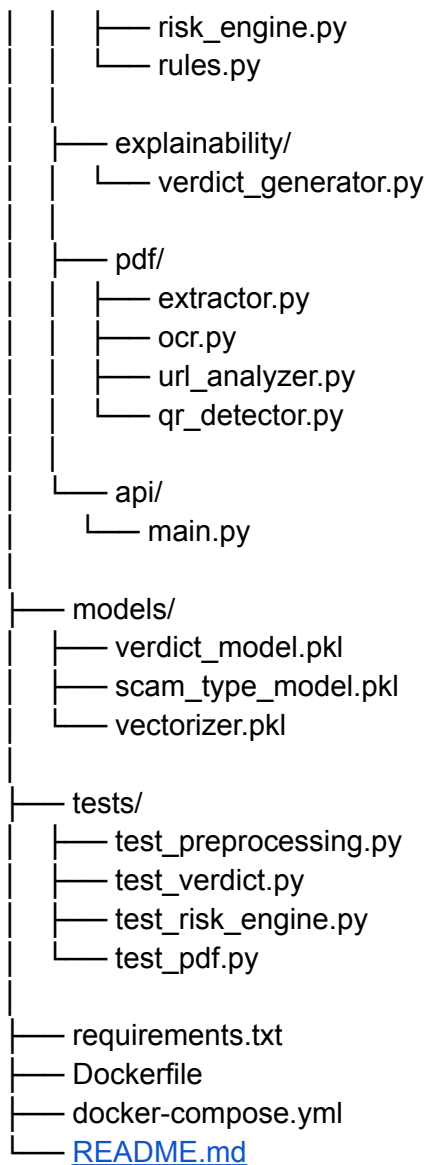
If that entire process works automatically, **Module 1 is complete.**

Recommended Final Project Structure

I would structure your GitHub repository like this:

scam-detection-chatbot/





The exact order I recommend you follow

Don't start all modules simultaneously.

Phase 1 — ML foundation

Module 1: Dataset validation

↓

Module 2: NLP preprocessing

↓

Module 3: Verdict classifier

↓

Module 4: Compare LR/SVM/RF/NB

↓

Module 5: Scam-type classifier

Phase 2 — Intelligence layer

Module 6: Risk Engine

↓

Module 7: Explainable Verdict

At this point you should have a **working local scam detector**.

Phase 3 — Productization

Module 8: FastAPI

↓

Module 9: WhatsApp integration

At this point you have a **working WhatsApp scam detector**.

Phase 4 — PDF intelligence

Module 10: PDF extraction

↓

Module 11: PDF URL/QR/OCR analysis

↓

Module 12: Unified PDF risk engine

Phase 5 — Research/evaluation

Module 13: Complete evaluation

↓

Module 14: End-to-end testing

↓

Module 15: Deployment + documentation

One architectural decision I strongly recommend

Your project should have **three separate layers**:

